

Module 5 Report

Software Assurance + Secure SQL (SQLi Defense)

Course: EN.605.256.82.FA26

This report documents the required reproducible setup, packaging, dependency analysis, SQL injection defenses, LIMIT controls, least-privilege database configuration, Snyk scan, testing, linting, and GitHub Actions continuous integration for Module 5.

Final verification

Requirement	Final evidence
Pytest	95 passed; 100.00% total coverage
Pylint	10.00/10
Snyk	49 dependencies tested; 0 issues
Database hardening	Least-privilege module5_app role verified
Dependency analysis	dependency.svg generated with pydeps + Graphviz
GitHub Actions	Successful CI run: Pylint, graph, Snyk, Pytest

1. Reproducible Environment: pip + uv

The project can be recreated from a clean Python environment using either the required pip + venv path or the alternate uv path. The README documents both methods.

pip + venv

```
python3 -m venv venv
source venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
pip install -e .
python -m pytest
python -m src.app
```

uv

```
python3 -m venv venv
source venv/bin/activate
python -m pip install uv
uv pip sync requirements.txt
uv pip install -e .
python -m pytest
python -m src.app
```

The requirements file includes the application dependencies as well as the required tooling, including Pylint and pydeps. Keeping the dependency list explicit reduces environment drift and makes local and CI execution reproducible.

2. Packaging with setup.py

The project includes setup.py so Module 5 can be installed as a Python package. Packaging gives imports a consistent structure across local runs, tests, and CI. Editable installation with `pip install -e .` allows source changes to be tested without repeatedly reinstalling the package. This reduces path-related 'works on my machine' problems and improves reproducibility. It also gives environment tools a standard installable project structure.

3. Dependency Graph

The dependency graph was generated with pydeps and Graphviz and is submitted separately as **dependency.svg**. The graph itself is also shown below for convenient review.

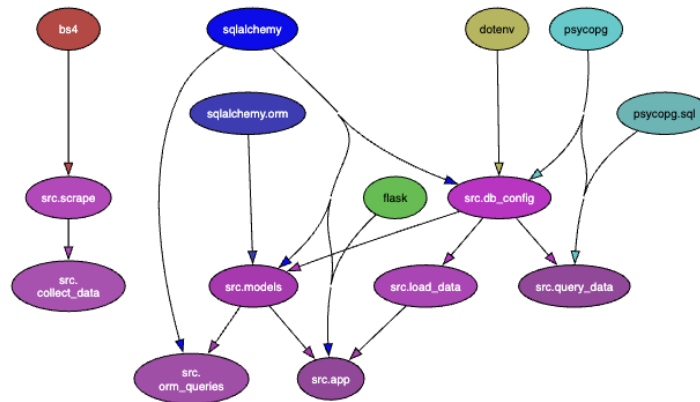


Figure 1. Module 5 dependency graph generated with pydeps + Graphviz.

Dependency graph summary (7 sentences)

The graph shows the relationships among the Module 5 source modules and key third-party dependencies. **src.app** contains the Flask application and interacts with the database and loading layers. **src.models** provides the SQLAlchemy model and uses the centralized **src.db_config** configuration. **src.db_config** centralizes PostgreSQL connectivity and is used by **src.load_data**, **src.query_data**, and **src.models**. **src.orm_queries** performs ORM-based analysis using SQLAlchemy and the Applicant model. **src.scrape** supports the data-collection portion of the application. Overall, the graph demonstrates separation among web, database configuration, persistence, querying, and scraping responsibilities.

```
pydeps src --noshow -T svg -o dependency.svg --max-bacon=2
```

4. SQL Injection Defenses and LIMIT Enforcement

SQL that accepts variable input was refactored so user-controlled values are never inserted directly into SQL text. Dynamic statements use `psycpg` composition with `sql.SQL`; dynamic identifiers use `sql.Identifier` where applicable; and data values are passed separately using placeholders and the parameter argument to `cursor.execute(statement, parameters)`. Statement construction is therefore separated from execution and parameter binding. The project does not use f-strings, string concatenation, or `.format()` to inject user values into raw SQL. A malicious-input test verifies that an SQL-injection-style value remains a parameter rather than becoming executable SQL, preventing unintended data exposure.

LIMIT safety control

Every query has an inherent LIMIT. Single-result and aggregate queries use LIMIT 1. Queries that accept a requested result count use a helper that clamps the value to the allowed range of 1 through 100, preventing oversized requests from bypassing the resource-control boundary.

```
safe_limit = clamp_limit(requested_limit)  # range: 1..100
stmt = sql.SQL("SELECT ... LIMIT %s")
cursor.execute(stmt, (safe_limit,))

stmt = sql.SQL("SELECT ... WHERE url = %s LIMIT 1")
cursor.execute(stmt, (url,))
```

5. Database Hardening and Least Privilege

Database credentials are loaded from environment variables rather than hard-coded in source: **DB_HOST**, **DB_PORT**, **DB_NAME**, **DB_USER**, and **DB_PASSWORD**. The project includes `.env.example` with placeholder values, while the real `.env` is excluded through `.gitignore`.

The application role **module5_app** follows least privilege. It is not a superuser, cannot create roles or databases, cannot create objects in the public schema, and is not the owner of the applicants table. It has SELECT and INSERT because the application reads applicant records and its loading path writes records. UPDATE and DELETE are not granted. Sequence USAGE is granted only so INSERT operations can obtain generated primary-key values.

```
GRANT CONNECT ON DATABASE module3_db TO module5_app;
GRANT USAGE ON SCHEMA public TO module5_app;
GRANT SELECT, INSERT ON TABLE applicants TO module5_app;
GRANT USAGE ON SEQUENCE applicants_p_id_seq TO module5_app;
```

Database privilege verification

```
Terminal Local x + v

(venv) sayedsayed@Sayeds-MacBook-Air module_5 % >...
has_table_privilege('module5_app','applicants','SELECT') AS can_select,
has_table_privilege('module5_app','applicants','INSERT') AS can_insert,
has_table_privilege('module5_app','applicants','UPDATE') AS can_update,
has_table_privilege('module5_app','applicants','DELETE') AS can_delete
FROM pg_roles r
WHERE r.rolname = 'module5_app';
"

```

	rolname	superuser	create_role	create_database	can_select	can_insert	can_update	can_delete
module5_app	f	f	f	t	t	f	f	

```
(1 row)
```

```
(venv) sayedsayed@Sayeds-MacBook-Air module_5 % psql -P pager=off -d module3_db -c "
SELECT
    r.rolname,
    r.rolsuper AS superuser,
    r.rolcreatorole AS create_role,
    r.rolcreatedb AS create_db,
    has_table_privilege('module5_app','applicants','SELECT') AS select_ok,
    has_table_privilege('module5_app','applicants','INSERT') AS insert_ok,
    has_table_privilege('module5_app','applicants','UPDATE') AS update_ok,
    has_table_privilege('module5_app','applicants','DELETE') AS delete_ok,
    has_schema_privilege('module5_app','public','CREATE') AS schema_create
FROM pg_roles r
WHERE r.rolname='module5_app';
"

```

	rolname	superuser	create_role	create_db	select_ok	insert_ok	update_ok	delete_ok	schema_create
module5_app	f	f	f	t	t	f	f	f	

```
(1 row)
```

```
(venv) sayedsayed@Sayeds-MacBook-Air module_5 %
```

Figure 2. Verification that module5_app is not a superuser and has SELECT/INSERT but not UPDATE, DELETE, or schema CREATE privileges.

6. Pylint and Automated Tests

Pylint was run on all Python source files under module_5/src. The documented final command disables the duplicate-code refactoring message for intentionally similar analysis queries while retaining the required 10/10 fail-under threshold. The final score is **10.00/10** with no displayed lint messages.

```
python -m pylint src --disable=duplicate-code --fail-under=10
```

Your code has been rated at 10.00/10

The final Pytest verification completed successfully with **95 passed** and **100.00% total coverage**.

7. Snyk Dependency Security Analysis

Snyk was used to scan the Python dependency set for known open-source vulnerabilities. The final scan tested **49 dependencies** and reported **0 total issues** and **no vulnerable paths found**. No dependency vulnerabilities therefore required remediation for this submission.

```
(venv) sayedsayed@Sayeds-MacBook-Air module_5 % snyk test
```

```
Testing (requirements.txt) ...
```

```
✓ Tested 49 dependencies for known issues, no vulnerable paths found.
```

Test Summary

Organization:	ssayed3
Test type:	open-source
Project path:	/Users/sayedsayed/jhu_software_concepts/module_5
Total issues:	0

💡 Tip

To view ignored issues, use the `--include-ignores` option.

Figure 3. Required snyk test evidence: 49 dependencies tested and 0 issues found.

8. GitHub Actions CI

The repository contains `.github/workflows/ci.yml`, configured to run on pushes and pull requests. The workflow installs Python dependencies and Graphviz, runs Pylint with the 10/10 fail-under threshold, generates and validates `dependency.svg`, runs the Snyk dependency scan, and executes Pytest. A PostgreSQL service is provided to the CI job for database-dependent tests.

Successful CI evidence

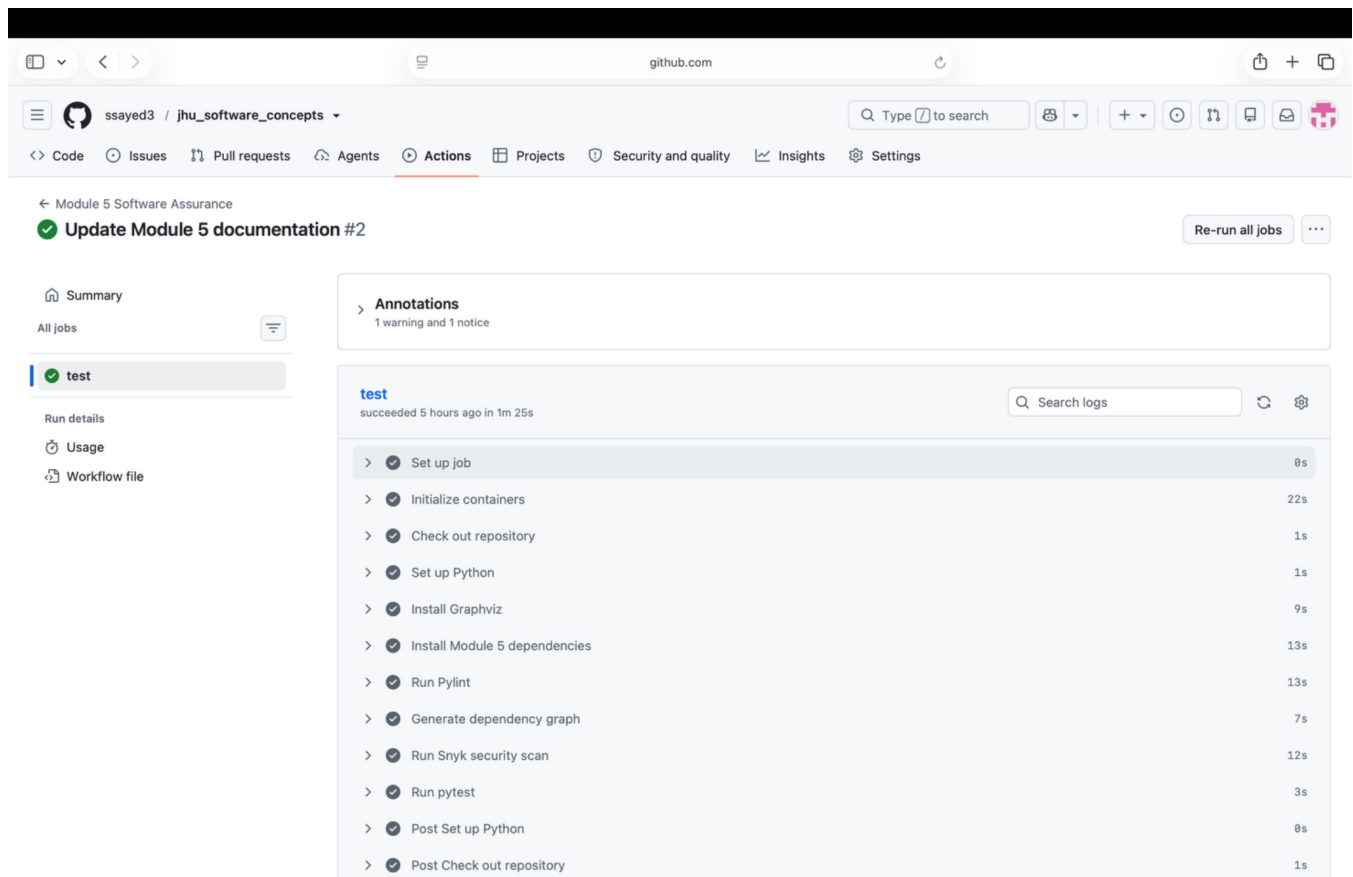


Figure 4. Successful GitHub Actions run showing the required Pylint, dependency graph, Snky, and Pytest steps.

9. Final Deliverables

The Module 5 submission includes the source and tests, README.md, requirements.txt, setup.py, .env.example, pytest.ini, dependency.svg, snyk-analysis.png, lint and coverage evidence, the successful CI screenshot, database permission evidence, and this module_5_report.pdf. The Canvas submission should contain a clean zipped module_5 folder matching the version committed to the course GitHub repository.

Optional extra credit: Snky Code (SAST) is not claimed in this report.